Parallel Implementation of Miniz Compression Using OpenMP

Vincenzo Gargano 667591 SPM Course a.a. 24/25

April 2025

1 Introduction

This report contains a description of the implementation of a parallel file compression and decompression system based on the given sequential implementation, using `DEFLATE` compression algorithm. The parallelization is obtained through OpenMP. Since the compression phase is the operation taking most time, this report will mostly focus on optimizing its steps. After a description of the methods and strategies used to parallelize, a performance analysis in terms of **strong speedup** will be presented in order to understand when the parallelization is effective.

2 Parallelization Strategy

The implementation follows a hybrid strategy that adapts to different file sizes. For small files, we defaults to sequential processing to avoid the overhead of parallelization. For larger files, the file is divided into multiple segments that can be processed independently in parallel.

In short, the program is divided into three parts.

1. Collection: Folders and subfolders are traversed to collect the files to compress.
2. Tasks Creation: `doWork_par` which will start the compression or decompression in parallel.
3. Tasks Work: The real core of parallelization: `compressData_par`.

The implementation takes advantage of two features offered by OpenMP: Task and Nested parallelism. The task parallelism is used for the **outer region**: this region spawns a team of threads, where the first thread entering the `omp single` directive starts creating tasks (`doWork_par`) that are then enqueued in the task queue and the remaining threads will pick up the tasks and execute. In this way we are parallelizing the directory traversal. A task start with directory traversal and ends with file compression. Whenever the file is big enough we would like to parallelize the compression and decompression of it, too. Nested parallelism is used to achieve this goal. The nested parallelism is used for the **inner region**, which is the one where effectively compression/decompression happens (`compressData_par` and `decompressData_par`). The inner region is parallelized using OpenMP's `omp parallel for` directive over chunks, which allows for efficient distribution of work among threads. The implementation uses dynamic scheduling to balance the workload across threads, ensuring that all threads are kept busy even if some chunks take longer to process than others.

2.1 Parallelism Structure: Outer Layer

In the example below, we focus on the relevant part of the code that implements the outer parallel region. Inside the outer region the `walkDir_par` function is called, this function recursively call itself taking subdir as firstprivate since we want different threads to work on different subdirectories. When a file is found `walkDir` will call the `doWork_par` function executing the compression.

```
#pragma omp parallel
{
    #pragma omp single
    {
        // Create tasks for each file in the directory
        for (const auto& file : files) {
            #pragma omp task firstprivate(subdir)
            {
                // Process the file
                bool subdir_result = walkDir_par(subdir, comp);
            }
        }
    }
    // Wait for all tasks to finish
    #pragma omp taskwait
}
```

2.2 Parallelism Structure: Inner Layer

Since this region is contained in a task, it will be executed by the thread that picks up the task from the queue, for OpenMP this part is sequential, but if we use `omp_set_nested(1)` with current nested level, we can nest the parallel region without waiting for the tasks to finish. So the thread executing that taks becomes the **master thread** of that task and here we can use the `omp parallel` directive to create a new team of threads working on chunk compression. We use dynamic scheduling with task chunk size of 1, allowing for efficient load balancing.

```
// Save current OpenMP state
int prev_nested = omp_get_nested();
int prev_max_active_levels = omp_get_max_active_levels();

// Enable nested parallelism
omp_set_nested(1);
omp_set_max_active_levels(2);

   #pragma omp parallel num_threads(thread_count)
    {
        #pragma omp for schedule(dynamic, 1)
        for (int i = 0; i < num_chunks; i++) {
            size_t offset = i * optimal_chunk_size;
            size_t current_chunk_size = std::min(optimal_chunk_size, size -
    offset);
            // Compress the chunk using pre-allocated buffer
            if (compress(compressed_chunks[i].data(), &compressed_sizes[i],
                         inPtr + offset, current_chunk_size) != Z_OK) {
                #pragma omp critical
                {
                    success = false;
                }
            }
        }
    }
// Restore previous OpenMP state
omp_set_nested(prev_nested);
omp_set_max_active_levels(prev_max_active_levels);
```

2.3 File Compression: Chunk-Based Parallelization

A chunk-based approach to divide the file into smaller segments that can be processed independently in parallel.

- If the chunk size is less than specified minimum size (`CHUNK_SIZE`: 1 MB), we will not parallelize the compression and will process the file sequentially.
- The chunk size is calculated considering both the file size and the number of available threads. It's preferred to have more chunks than threads to improve load balance and avoid oversubscription.

The chunking strategy is designed to divide the file in equal parts, to achieve this we calculate the optimal chunk size as shown in the code below. For example, if we have `CHUNK_SIZE` 1MB and 8 threads, with 10MB file to compress, and we set the number of chunks to `TARGET_CHUNKS=threads*4` each chunk will be around $\frac{10MB}{8*4} = 320KB$. This can be avoided to set a max between the chunk size obtained and the `CHUNK_SIZE`. Instead if we have a 100MB file, the chunk size will be $\frac{100MB}{8*4} = 3.125MB$, which will be the size of each chunk.

```
// Create more chunks than threads to improve load balancing
size_t min_chunk_size = BUF_SIZE; // Minimum 1MB chunks
int target_chunks = num_threads * 4; // 4 chunks per thread for better load
    balancing

// Calculate chunk size based on file size and target number of chunks
size_t optimal_chunk_size = std::max(min_chunk_size, size / target_chunks);
int num_chunks = (size + optimal_chunk_size - 1) / optimal_chunk_size;
```

2.4 File Decompression

For the File decompression, we read the metadata in the header of compressed file, calculate the offsets for the chunks and use a decompression parallel loop for writing the chunks in the right part of output file.

2.4.1 Other minor optimizations

Pre-allocation of Resources: To minimize memory allocation overhead during parallel execution, we can pre-allocates vectors for compressed chunks and their respective sizes before entering the parallel region:

```
// Pre-allocate vectors with the right size to avoid reallocations within
    parallel region
std::vector<std::vector<unsigned char>> compressed_chunks(num_chunks);
std::vector<size_t> compressed_sizes(num_chunks);

// Pre-allocate storage for each chunk's compressed data
for (int i = 0; i < num_chunks; i++) {
    size_t chunk_offset = i * optimal_chunk_size;
    size_t current_chunk_size = std::min(optimal_chunk_size, size - chunk_offset
    );
    size_t bound = compressBound(current_chunk_size);
    compressed_chunks[i].resize(bound);
    compressed_sizes[i] = bound;  // Will be updated in parallel region
}
```

3 Performance Evaluation

For the performance evaluation, both implementations were tested on three types of datasets, the data used were randomly generated hence they have maximum entropy and sometimes the compressed file is bigger than the original one. The datasets are as follows:

- **Many small files**: A collection of 777 small files, each with a size of **16 KB**, for a total of **13 MB**
- **Few large files**: A directory containing 3 large files, each with a size of **100 MB** for a total of **300 MB**
- **Mixed-size files**: A directory containing files of varying sizes 100 small sized (**16 KB**), 10 medium sized (**20 MB**) and 3 big sized (**100 MB**) totaling around **502 MB**.

The test were performed on the SPM cluster machine, having a NUMA architecture with 32 physical cores. In order to execute all the test the command `sbatch` was used to submit the job to the cluster. For a time stable result the script execute n times and compute an average of the time. The command used for executing each test is this:

```
./minizpar -q 2 -r 1 -C 0 -s 1 -t n_threads ../directory_to_compress
```

The parameters added are `-s n` setting the minimum multiplier for chunk size where n is the multiplier, `-t n` setting the number of threads to use, the other flags are equivalent to the ones used in the sequential version.

3.1 Strong Speedup

The strong speedup is defined as the ratio of the execution time of the sequential version to the execution time of the parallel version, both processing the same input data. The evaluation shows the speedup as number of threads increases.

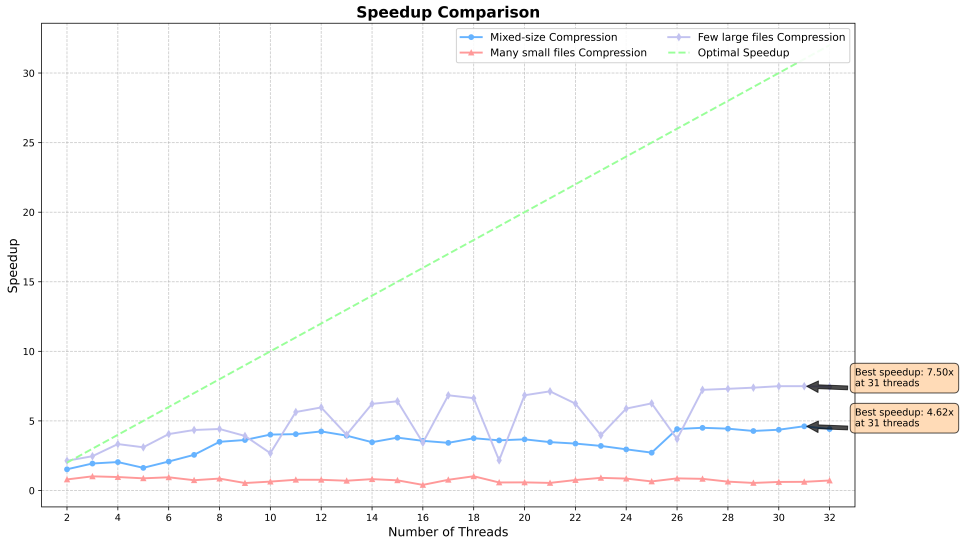

Figure 1: Performance comparison between sequential and parallel implementations

Starting with the **Few large files Compression** we can see in fig **??** that speedup in general is *sublinear*, this happens because we are on a **NUMA architecture** and we don't have data locality. This means that the threads executing on a certain node could be working on data that is not local to that node, causing a performance penalty due to memory access latency.
For **Many small files Compression**: single files are compressed sequentially (no chunks) by

a team of threads in the outer layer, increasing latency as the number of threads grows. With two threads we get a slight increase in speed, but then peformance decreases with more threads. Finally for **Mixed-size Compression** we get a speedup of $\times 4.62$ with a trend similar to **Few large files**, the loss in speedup happens since there are more files and the small ones have to be processed

3.2 Efficency and Amhdal's Law

If we take a look at the efficency of the parallel implementation, $E = Speedup/n_{threads}$, we notice a downward trend. As number of threads increase we start to hit diminishing returns in performances.

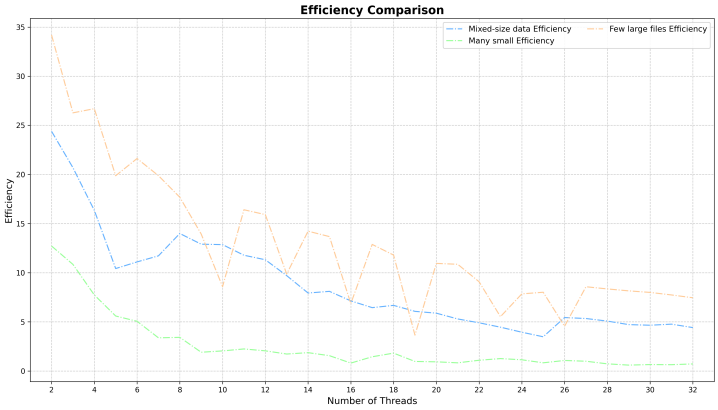

Figure 2: Efficiency plot of the conducted test

Since I/O bound operations are inherently sequential, and we know that efficiency is influenced by the amount of sequential computation. If we try to calculate for one case, the general **Mixed-size data Compression**. We can only estimate the sequential part of the code and we're compressing files sequentially in parallel, so we don't know the fraction of the code that is sequential.

3.2.1 Theoretical speedup

If we try different sequential fractions, we can see how in those cases the theoretical speedup changes. Assuming we use 32 threads we get the following:

$$Speedup = \frac{1}{s + \frac{1-s}{N}} \tag{1}$$

Where s is the fraction of the code that is sequential and N is the number of threads.

Table 1: Theoretical Speedup vs. Sequential Fraction (32 Threads)

Sequential Fraction (s)	**Theoretical Speedup**
1%	24.4×
5%	12.5×
10%	7.8×
20%	4.4×

If we want align with our actual results of **Mixed-size Compression** we can assume a sequential fraction between 5% and 10%, this means that the theoretical speedup is between $12.5\times$ and $7.8\times$, which is consistent with the observed speedup of $4.62\times$. An analogous reasoning can be applied with the other cases.

3.3 Decompression Performance

Briefly speaking about decompression, the maximum speedup we can get with 8 threads is 2.2x, up to 32 threads we get a speedup of 3x, this is because the decompression is not CPU bound, but I/O bound.

4 Conclusion

The parallel implementation of the Miniz compression library using OpenMP demonstrates effective use of parallelization techniques to improve performance, combining sequential processing for small files with parallel processing for larger files, optimizes resource utilization while minimizing overhead. The chunk-based strategy for large files, coupled with dynamic scheduling and pre-allocation of resources, provides an efficient framework for parallel compression and decompression. The most difficult part of the assignment was understading the OpenMP tasking internal model, if we use tasks we can end up obtaining a sequential execution of what we think is parallel.
OpenMP offers a lot of flexibility, in fact we could implement things differently: for example doing the first pass of directory traversal and then after all files are found we could have parallelized the compression with the `omp parallel for` directive.
Finally we can conclude that using OpenMP is a very fast way to write parallel code with an high level abstraction over the synchronization and thread management, we reduced the compression of a 500MB files from sequential version taking 37.5 seconds to the parallel implementation taking around 8.3 seconds with 32 physical cores. Future work could focus on further optimizing the chunk size selection algorithm from the test the best size was around 1 to 3, and the main source of speedup can be also obtained by using OpenMP thread affinity to bind threads to specific cores, reducing the overhead of thread migration and improving cache locality.